语言模型（三）-语言模型的演进

- 来源: https://class.imooc.com/lesson/5255#mid=104879
- 阶段: 阶段一: 【认知基石】理解大模型的本质与工作原理
- 周: 第1周 语言模型是什么
- 模块: 语言模型
- 课节: 图文 1-5 语言模型（三）-语言模型的演进

1-5 语言模型（三）-语言模型的演进

语言模型（三）-语言模型的演进

> [!TIP] 阅读指南 上一节，我们了解了语言模型的规律是从数据中学出来的。语言模型是怎么一步步发展到现在这种程度的？早期模型也能学到规律吗？ 本节将回顾语言模型的三代演进史，看它如何从只能记住3个词，进化到能理解整本书。

10.1 第一代：N-gram模型（1990年代-2000年代）

最早的语言模型很"笨"：它只看前面几个词。

比如，一个"2-gram"（bigram）模型：

$P(w2 \mid w1) = \text{count}(w1, w2) \text{ count}(w1)$ P(w 2 ∣w 1)= count(w 1) count(w 1 ,w 2)

w1

w2

这个公式的意思是：

- 分子：在训练数据中，"很好"这个组合出现了多少次
- 分母：在训练数据中，"很"这个字出现了多少次
- 结果："很"后面跟"好"的概率

举个具体例子，如果训练数据中：

> "很"出现了1000次 "很好"出现了350次

那么：

$P(w2 \mid w1) = 350/1000 = 0.35 = 35\%$ P(w 2 ∣w 1)= 1000 350 =0.35=35%

但这是有问题的。

想象一个句子：

> “雨停了，太阳穿过云层撒在湿漉漉的街道上。小女孩蹲下来，仔细看着水洼里的____”

要预测下一个词，需要知道“水洼”和前面的“雨停了”有关。但“雨停了”在十几个词之前！一个只看前2-3个词的模型，根本记不住这么远。

就像一个记忆只有3秒的人，无法理解复杂的对话。

10.2 第二代：RNN/LSTM模型（2010年代早期）

循环神经网络（RNN）试图解决这个问题：用一个“隐藏状态”来记忆历史信息。

RNN的核心思想

想象一个人在读一篇文章，边读边做笔记：

> 读第1个词 → 在笔记本写下要点 → 记忆状态$_1$ 读第2个词 → 结合记忆状态$_1$ → 更新笔记 → 记忆状态$_2$ 读第3个词 → 结合记忆状态$_2$ → 更新笔记 → 记忆状态$_3$ ...

每个词都在前一个记忆的基础上更新，理论上能记住整个历史。

信息衰减问题

理论上，RNN可以记住整个历史。但实际上，信息在传递过程中会衰减，就像传话游戏：

> 第1个人：“明天早上8点在学校门口集合，带上雨伞和水壶” 第2个人：“明天早上8点在校门口集合，带雨伞和水壶” ← 丢了“学校” 第3个人：“明天早上8点在校门集合，带雨伞” ← 丢了“水壶” 第10个人：“明天见” ← 几乎所有细节都没了

因为每经过一层，信息都要乘以一个小于1的系数（比如0.9）： $0.9^{10} \approx 0.35$ $0.9^{100} \approx 0.00003$

传到后面，信息几乎就消失殆尽了。

LSTM的改进

为了解决这个问题，研究者发明了LSTM（长短期记忆网络），增加了“遗忘门”、“输入门”、“输出门”。就像一个智能笔记本：遗忘门决定哪些旧信息可以忘掉，输入门决定哪些新信息值得记住，输出门决定当前应该关注什么。

效果是能记住更长的历史，从10-20个词提升到100-200个词。但即使是LSTM，在真正的长文本（数千个词）面前，仍然力不从心。

就像一个记忆只有20分钟的人，只能看懂一部动漫，却无法理解一部两小时的电影。

10.3 第三代：Transformer模型（2017年至今）

从“传话”到“群聊”

2017年，Transformer横空出世。它用Self-Attention机制，彻底解决了长程依赖问题。

Transformer不再像以前的模型依赖“传话”的模式，它革命性地把“传话”模式改为了“群聊”：

> 10个人在一个微信群里： 小明在群里说：“明天下午3点在图书馆见，记得带笔记本” ↓ 所有人都能直接看到 小红、小李、小王、...、小张：都能直接看到完整消息！ 每个人想回复时，可以直接看到小明说了什么，可以看到所有人的发言。信息不会丢失，每个人都有完整的上下文。

更进一步，Transformer不仅让所有词互相“看到”，还能让每个词智能地“关注”重点：

句子：“昨天我在公园里遇到了多年未见的老朋友张三，他现在在上海工作，他很开心” 当处理最后的“他”时：

- RNN：只能依稀记得前面提到过“张三”
- Transformer：能直接“看到”整个句子，并且智能地把注意力集中在“张三”上

注意力分配（简化示意）：

```
┌──────────────────────────────┐
│ "他"对各个词的注意力权重：
├──────────────────────────────┤
│ 昨天：     2%  ← 不太相关
│ 公园：     1%
│ 老朋友：   8%
│ 张三：    70%  ← 高度相关！
│ 上海：     5%
│ 工作：    10%
│ 很：       2%
│ 开心：     2%
└──────────────────────────────┘
```

Self-Attention的优势

- 无损传递：任何两个词之间都是直接连接，不需要经过中间层。
- 智能筛选：自动计算哪些词重要，把注意力放在关键位置。
- 并行计算：所有词可以同时处理，速度快。无限长度（理论上）：不管句子多长，每个词都能“看到”所有其他词。

回到刚才的例子：

> “昨天我在公园里遇到了多年未见的老朋友张三，他看起来很开心，现在在上海工作，他_____”

Transformer在预测最后的空白时，可以直接“注意到”“张三”这个名字（指代消解），直接“注意到”“很开心”这个情绪词（情感理解），直接“注意到”“上海工作”（上下文信息）。

所以它能准确预测：“很满意现在的生活”、“经常回来看看”等合理的续写。

ChatGPT就是基于Transformer的语言模型。

这也是为什么ChatGPT能够理解长篇幅的上下文（GPT-4支持128,000个token），在对话中记住之前说过的内容，处理复杂的逻辑推理，准确理解指代关系。

因为它站在了三代语言模型演进的肩膀上，用Transformer彻底解决了RNN时代的“遗忘”问题。

从传话游戏到群聊，这不仅是技术的进步，更是范式的革命。

10.4 ■ 学点英语

中文	English	音标	说明
N元模型	N-gram Model	/ˈen græm ˈmɒdl/	基于前n-1个词预测第n个词的统计模型，上下文窗口固定
循环神经网络	Recurrent Neural Network (RNN)	/rɪˈkɜːrənt ˈnjʊərəl ˈnetwɜːk/	通过链式结构按顺序处理序列的网络，越长越容易遗忘
长短期记忆网络	Long Short-Term Memory (LSTM)	/lɒŋ ʃɔːt tɜːm ˈmeməri/	引入门控机制缓解信息衰减的改进型RNN
注意力机制	Attention Mechanism	/əˈtenʃən ˈmekənɪzəm/	让模型在处理序列时动态关注不同位置相关信息的机制
自注意力机制	Self-Attention	/self əˈtenʃən/	让序列中每个元素直接与所有其他元素交互的机制，Transformer的核心
缩放点积注意力	Scaled Dot-Product Attention	/skeɪld dɒt ˈprɒdʌkt ə ˈtenʃən/	Transformer中计算注意力权重的方式，通过Query和Key的兼容性决定Value的加权权重
梯度消失	Vanishing Gradient	/ˈvænɪʃɪŋ ˈgreɪdiənt/	深层网络中梯度随层数增加指数衰减导致早期层无法有效学习的现象
上下文窗口	Context Window	/ˈkɒntekst ˈwɪndəʊ/	模型在预测下一个Token时能参考的最大Token数量范围
Transformer架构	Transformer	/trænsˈfɔːrmər/	基于Self-Attention的架构，支持并行计算和长距离依赖建模